

Artificial Intelligence

Prof. Chang-Hee Han

8

Introduction to deep learning for computer vision

8.0 This chapter covers

▣ Today's lecture covers

- Understanding convolutional neural networks (convnets)
- Using data augmentation to mitigate overfitting

8.1 Introduction to convnets

▣ Introduction to convolutional neural networks (ConvNet)

- Let's learn what a Convolutional Neural Network (ConvNet) is and why it works so well for computer vision tasks.
- First, we will look at a simple example of a ConvNet.
- In Chapter 2, we solved the MNIST handwritten digit classification problem using a densely connected network (test accuracy: 97.8%).
- Now, we will try using a ConvNet for the same MNIST task.
- Even a basic ConvNet will perform much better than the fully connected model we used before.

8.1 Introduction to convnets

▣ Introduction to convolutional neural networks (ConvNet)

- Code 8-1 shows a basic example of a ConvNet.
- It stacks **Conv2D** and **MaxPooling2D** layers.
- We will build the model using the **Functional API** introduced in the previous chapter.

Listing 8.1 Instantiating a small convnet

```
from tensorflow import keras
from tensorflow.keras import layers
inputs = keras.Input(shape=(28, 28, 1))
x = layers.Conv2D(filters=32, kernel_size=3, activation="relu")(inputs)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=64, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=128, kernel_size=3, activation="relu")(x)
x = layers.Flatten()(x)
outputs = layers.Dense(10, activation="softmax")(x)
model = keras.Model(inputs=inputs, outputs=outputs)
```

8.1 Introduction to convnets

▣ Introduction to ConvNet

- It's important to note that a ConvNet uses an input tensor of shape (image_height, image_width, image_channels), excluding the batch dimension.
- In this example, the ConvNet is set up to handle input of shape (28, 28, 1), which is the format of MNIST images.

Listing 8.2 Displaying the model's summary

```
>>> model.summary()
Model: "model"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
input_1 (InputLayer)         [(None, 28, 28, 1)]       0
_________________________________________________________________
conv2d (Conv2D)              (None, 26, 26, 32)        320
_________________________________________________________________
max_pooling2d (MaxPooling2D) (None, 13, 13, 32)        0
_________________________________________________________________
conv2d_1 (Conv2D)            (None, 11, 11, 64)        18496
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (None, 5, 5, 64)          0
_________________________________________________________________
conv2d_2 (Conv2D)            (None, 3, 3, 128)         73856
_________________________________________________________________
flatten (Flatten)            (None, 1152)              0
_________________________________________________________________
dense (Dense)                (None, 10)                11530
=================================================================
Total params: 104,202
Trainable params: 104,202
Non-trainable params: 0
_________________________________________________________________
```

8.1 Introduction to convnets

▣ Introduction to ConvNet

- The output of Conv2D and MaxPooling2D layers is a rank-3 tensor with shape (height, width, channels).
- The height and width dimensions tend to get smaller as the model becomes deeper.
- The number of channels is controlled by the first argument of the Conv2D layer (e.g., 32, 64, or 128).
- The output shape of the last Conv2D layer is (3, 3, 128).
- This means the feature map has size 3×3 with 128 channels.
- The next step is to feed this output into a fully connected classifier.
- This classifier is made of stacked Dense layers, which we′ve already seen before.
- The classifier processes 1D vectors, but the previous layer outputs a rank-3 tensor.
- Therefore, we need a Flatten layer before the Dense layers to reshape the 3D output into a 1D tensor.

8.1 Introduction to convnets

▣ Introduction to ConvNet

- Finally, to classify the 10 classes, set the output size of the last layer to 10 and use the softmax activation function.
- Now, we will train this ConvNet on the MNIST digit images.
- We will reuse much of the MNIST example code from Chapter 2.
- Since the softmax activation outputs probabilities for 10 classes, we will use the categorical cross-entropy loss.
- Because the labels are integers, we will use the sparse categorical cross-entropy loss (sparse_categorical_crossentropy).

8.1 Introduction to convnets

▣ Introduction to ConvNet

- Let's train the convnet on the MNIST digits.

Listing 8.3 Training the convnet on MNIST images

```
from tensorflow.keras.datasets import mnist

(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
train_images = train_images.reshape((60000, 28, 28, 1))
train_images = train_images.astype("float32") / 255
test_images = test_images.reshape((10000, 28, 28, 1))
test_images = test_images.astype("float32") / 255
model.compile(optimizer="rmsprop",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.fit(train_images, train_labels, epochs=5, batch_size=64)
```

8.1 Introduction to convnets

▣ Introduction to ConvNet

- Let's evaluate the model on the test data.

Listing 8.4 Evaluating the convnet

```
>>> test_loss, test_acc = model.evaluate(test_images, test_labels)
>>> print(f"Test accuracy: {test_acc:.3f}")
Test accuracy: 0.991
```

- The fully connected network from Chapter 2 achieved 97.8% test accuracy, while even a basic ConvNet reached 99.1% test accuracy.
- The error rate was reduced by about 60% (relatively).

8.1 Introduction to convnets

8.1.1 The convolution operation

- The main difference between fully connected layers and convolutional layers is as follows:
  - A Dense layer learns global patterns in the input feature space (for example, patterns across all pixels in an MNIST digit image), while a convolutional layer learns local patterns.
  - For images, it looks for patterns in small 2D windows over the input (see Figure 8-1).
- In the previous example, all these windows had a size of 3×3.

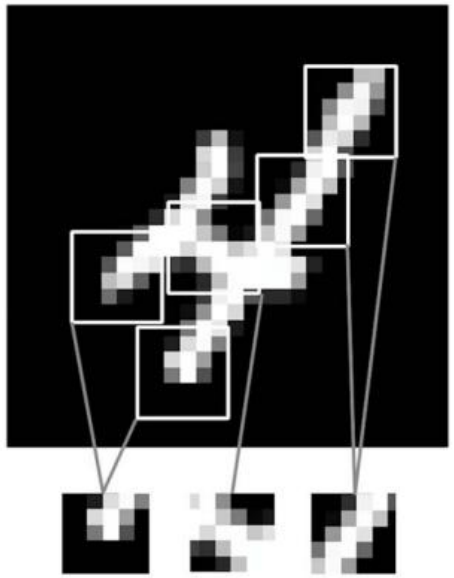


Figure 8.1 Images can be broken into local patterns such as edges, textures, and so on.

8.1 Introduction to convnets

8.1.1 The convolution operation

- This key feature gives ConvNets two interesting properties.
  - ➢ The learned patterns are translation invariant.
    - If a ConvNet learns a pattern in the bottom-right corner of an image, it can also recognize the same pattern in another place (for example, in the top-left corner).
    - A fully connected network, on the other hand, must learn the same pattern again when it appears in a new position.
    - This property allows ConvNets to process images efficiently (since the world we see doesn't change when things move around).
    - As a result, ConvNets can learn generalizable representations even with a small number of training samples.

8.1 Introduction to convnets

8.1.1 The convolution operation

- This key feature gives ConvNets two interesting properties.
  - ➢ A ConvNet can learn the spatial hierarchy of patterns.
    - The first convolutional layer learns small local patterns, such as edges.
    - The second convolutional layer learns larger patterns made up of the features from the first layer.
    - By using this approach, ConvNets can effectively learn complex and abstract visual concepts.
    - This works well because the world we see naturally has a spatial hierarchical structure.

8.1 Introduction to convnets

8.1.1 The convolution operation

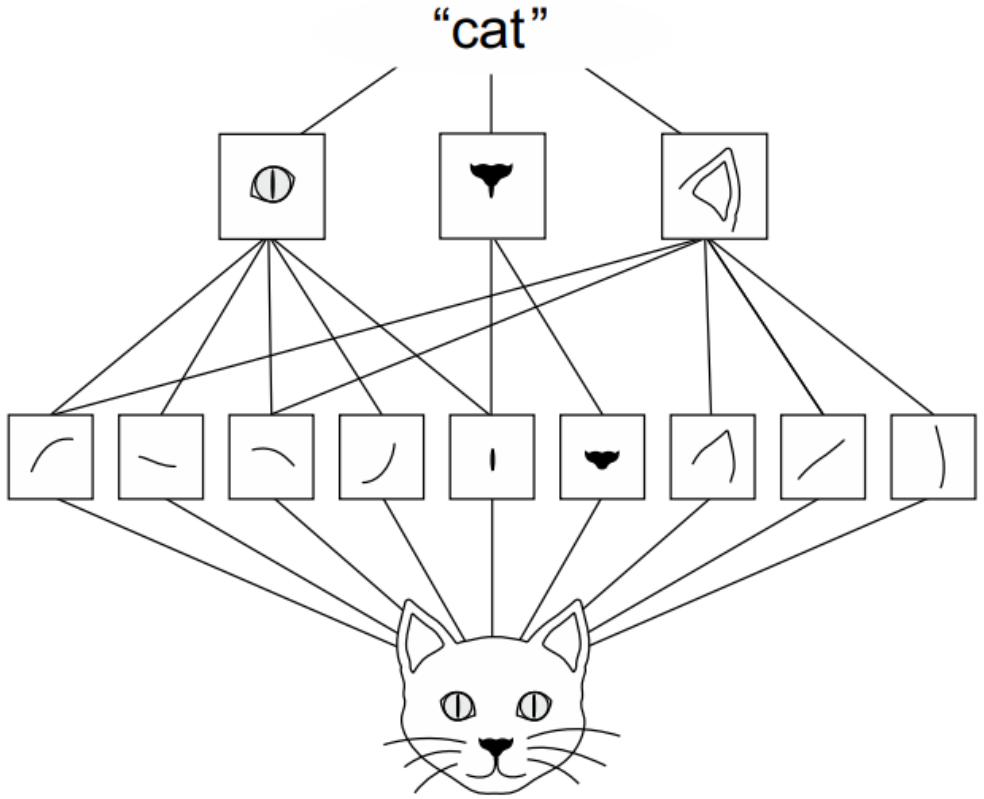


Figure 8.2 The visual world forms a spatial hierarchy of visual modules: elementary lines or textures combine into simple objects such as eyes or ears, which combine into high-level concepts such as "cat."

8.1 Introduction to convnets

▣ 8.1.1 The convolution operation

- A convolution operation is applied to a rank-3 tensor called a feature map.
- This tensor has two spatial axes (height and width) and a depth axis (also called the channel axis).
- An RGB image has three color channels (red, green, and blue), so its depth dimension is 3.
- A grayscale image, like those in MNIST, has a depth dimension of 1 (gray tone).
- The convolution operation extracts small patches from the input feature map, applies the same transformation to each patch, and produces an output feature map.

8.1 Introduction to convnets

▣ 8.1.1 The convolution operation

- The output feature map is also a rank-3 tensor with height and width.
- The depth of the output tensor depends on a parameter of the layer, so it can vary.
- This means that the depth channels no longer represent specific colors like in an RGB input.
- Instead, each depth channel represents a filter.
- A filter encodes a specific feature of the input data.
- For example, at a high level, one filter might encode whether there is a face in the input.
- In the MNIST example, the first convolutional layer takes an input feature map of shape (28, 28, 1) and outputs one of shape (26, 26, 32).
- This means 32 filters are applied to the input.
- Each of the 32 output channels contains an array of values with size 26×26.
- These values form the response map of the filters to the input.
- Each response shows how the filter pattern reacts at every position of the input.

8.1 Introduction to convnets

8.1.1 The convolution operation

- The meaning of the term feature map is as follows:
  - Each dimension along the depth axis represents one feature (or filter), and the rank-2 tensor output[:, :, n] is a 2D spatial map showing the response of that filter

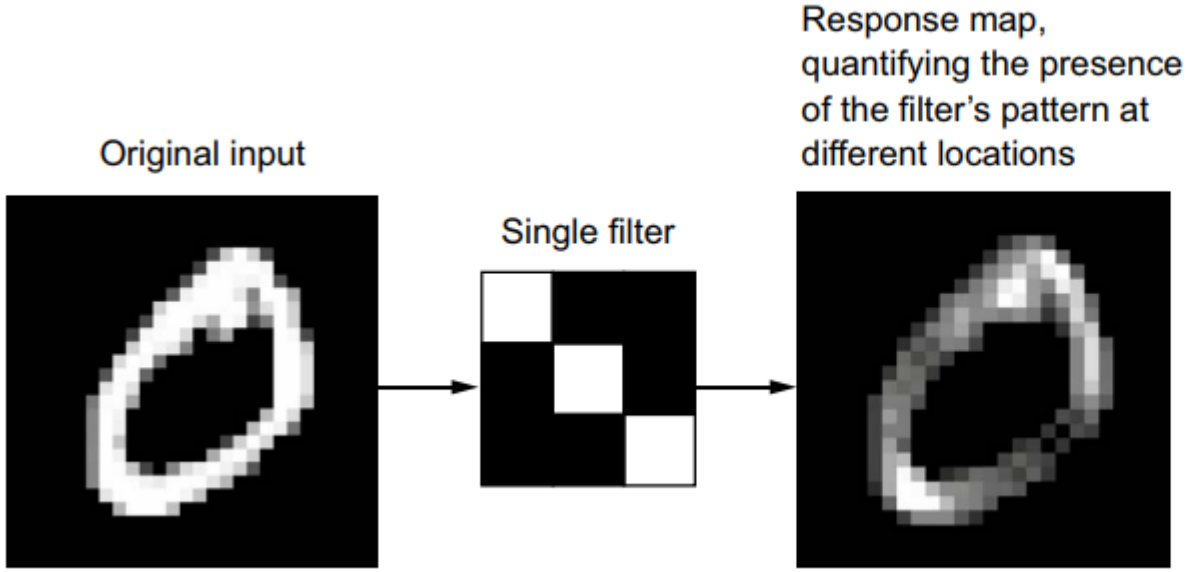


Figure 8.3 The concept of a response map: a 2D map of the presence of a pattern at different locations in an input

8.1 Introduction to convnets

▣ 8.1.1 The convolution operation

- A convolution operation is defined by two main parameters:
  - ➢ **Size of the patch extracted from the input:**
    - Typically 3×3 or 5×5 in size.
    - In this example, we use the commonly used 3×3 size.
  - ➢ **Number of filters** that will be computed to produce the **output feature map depth**:
    - In this example, it starts with a depth of 32 and ends with 128.
- In Keras, these parameters are passed to the Conv2D layer as the first and second arguments, like: Conv2D(output_depth, (window_height, window_width))

8.1 Introduction to convnets

8.1.1 The convolution operation

- The convolution works by sliding a 3×3 or 5×5 window over the 3D input feature map, extracting 3D patches of shape (window_height, window_width, input_depth) at every location.
- Each 3D patch is multiplied by a learned tensor of weights called a convolution kernel, producing a 1D vector of size (output_depth).
- The same kernel is applied repeatedly to all patches.
- All transformed vectors are then rearranged into a 3D feature map of shape (height, width, output_depth).
- Each spatial position in the output feature map corresponds to the same position in the input feature map (for example, the bottom-right output cell contains information from the bottom-right part of the input).
- When using a 3×3 window, the 3D patch input[i-1:i+2, j-1:j+2, :] produces the output vector output[i, j, :].

8.1 Introduction to convnets

8.1.1 The convolution operation

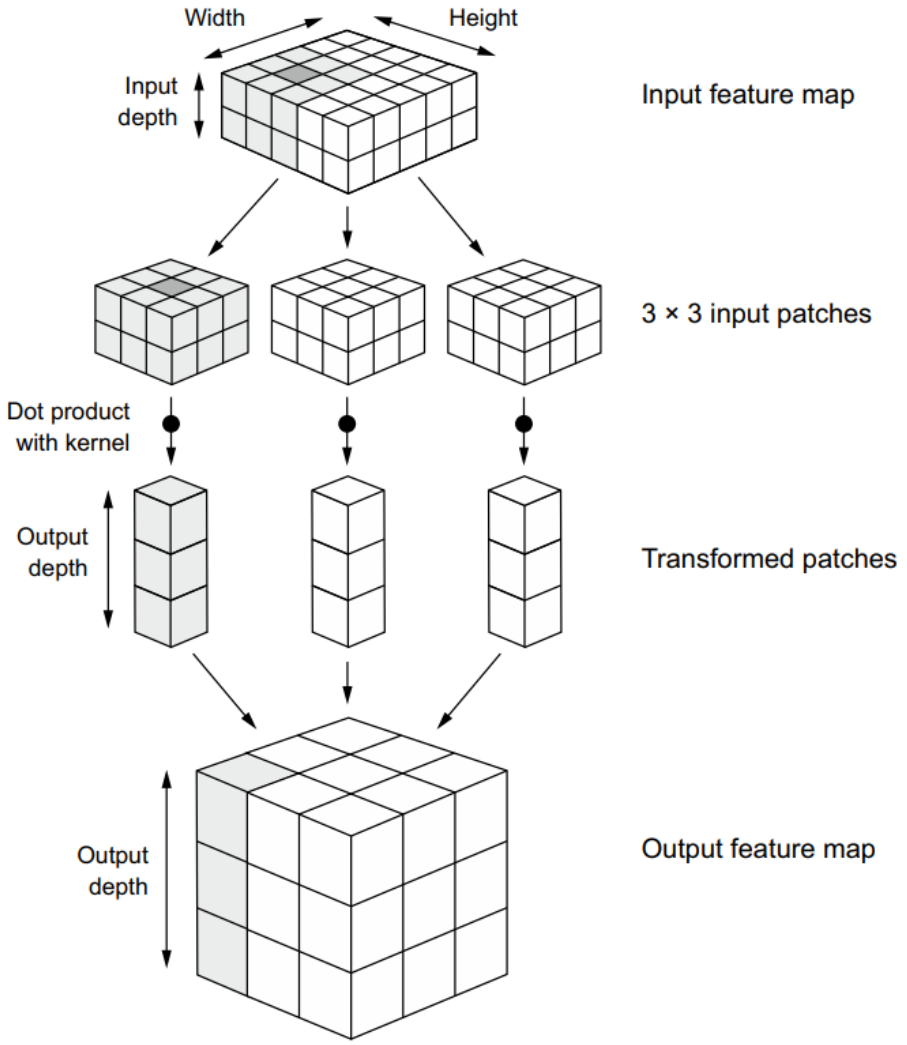


Figure 8.4 How convolution works

8.1 Introduction to convnets

▣ 8.1.1 The convolution operation

- For two reasons, the output height and width can be different from the input height and width:
  - ➢ **Boundary issue:** This can be handled by adding padding to the input feature map.
  - ➢ Depending on whether stride (which will be explained soon) is used or not.

8.1 Introduction to convnets

▣ 8.1.1 The convolution operation

- **Understanding Boundary Issues and Padding**
  - Let's think about a feature map of size 5×5 (so there are 25 tiles in total).
  - A 3×3 window can be centered only on tiles that form a 3×3 grid, meaning there are only 9 possible positions.
  - Therefore, the output feature map becomes 3×3 in size.
  - The overall size becomes slightly smaller.
  - In this case, both the height and width shrink by 2 tiles.
  - We already saw this kind of boundary issue in the earlier example.
  - In the first convolutional layer, the input size of 28×28 became 26×26.

8.1 Introduction to convnets

8.1.1 The convolution operation

- **Understanding Boundary Issues and Padding**

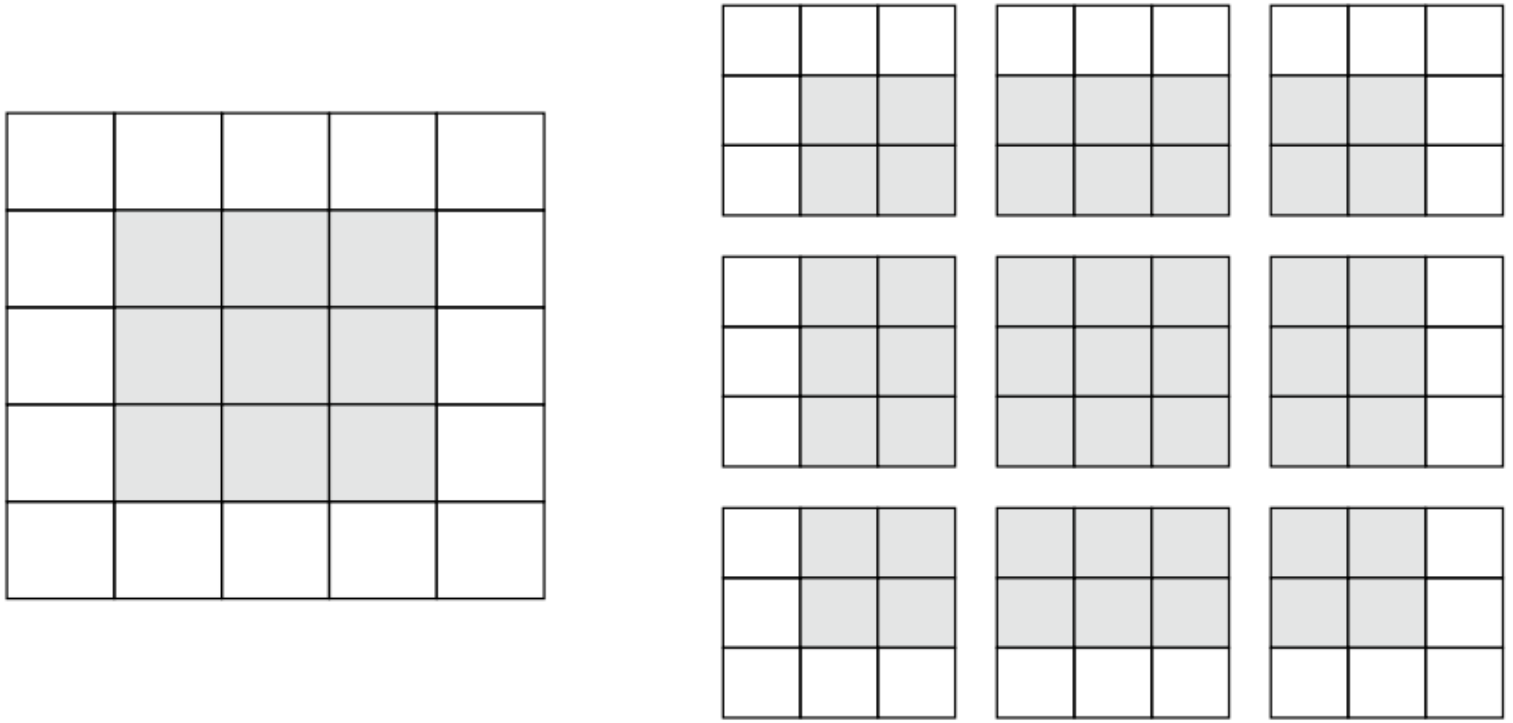


Figure 8.5 Valid locations of 3 × 3 patches in a 5 × 5 input feature map

8.1 Introduction to convnets

▣ 8.1.1 The convolution operation

- **Understanding Boundary Issues and Padding**
  - If you want the output feature map to have the same height and width as the input, you can use padding.
  - Padding adds a suitable number of rows and columns around the edges of the input feature map.
  - This allows the center of the convolution window to be placed over every input tile.
  - For a 3×3 window, add one row at the top and bottom, and one column on the left and right (see Figure 8-6).
  - For a 5×5 window, add two rows and two columns.

8.1 Introduction to convnets

8.1.1 The convolution operation

- **Understanding Boundary Issues and Padding**

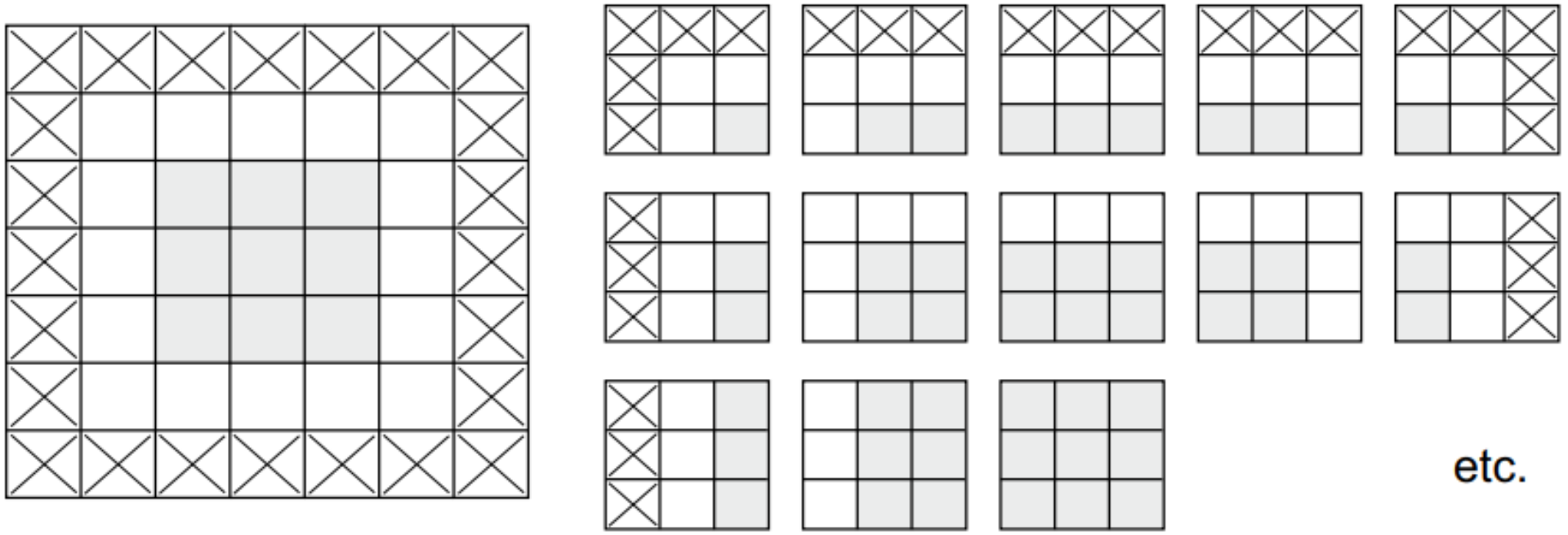


Figure 8.6 Padding a 5 × 5 input in order to be able to extract 25 3 × 3 patches

8.1 Introduction to convnets

▣ 8.1.1 The convolution operation

- In a Conv2D layer, padding can be set using the padding parameter.
- There are two possible values:
  - 'valid' means **no padding** is used (the window only slides over valid positions).
  - 'same' means **padding is added** to make the output have the **same height and width** as the input.
- The default value of the padding parameter is 'valid'.

8.1 Introduction to convnets

▣ 8.1.1 The convolution operation

- **Understanding Convolution Stride**
  - Another factor that affects the output size is the stride.
  - So far, we've assumed that the center of the convolution window moves one tile at a time across the input.
  - The distance between two consecutive window positions is a parameter called the stride.
  - The default stride value is 1.
  - Convolutions can also be performed with a stride greater than 1.
  - In Figure 8-7, you can see the patches extracted by convolving a 3×3 window with stride 2 on a 5×5 input (without padding).

8.1 Introduction to convnets

8.1.1 The convolution operation

- **Understanding Convolution Stride**

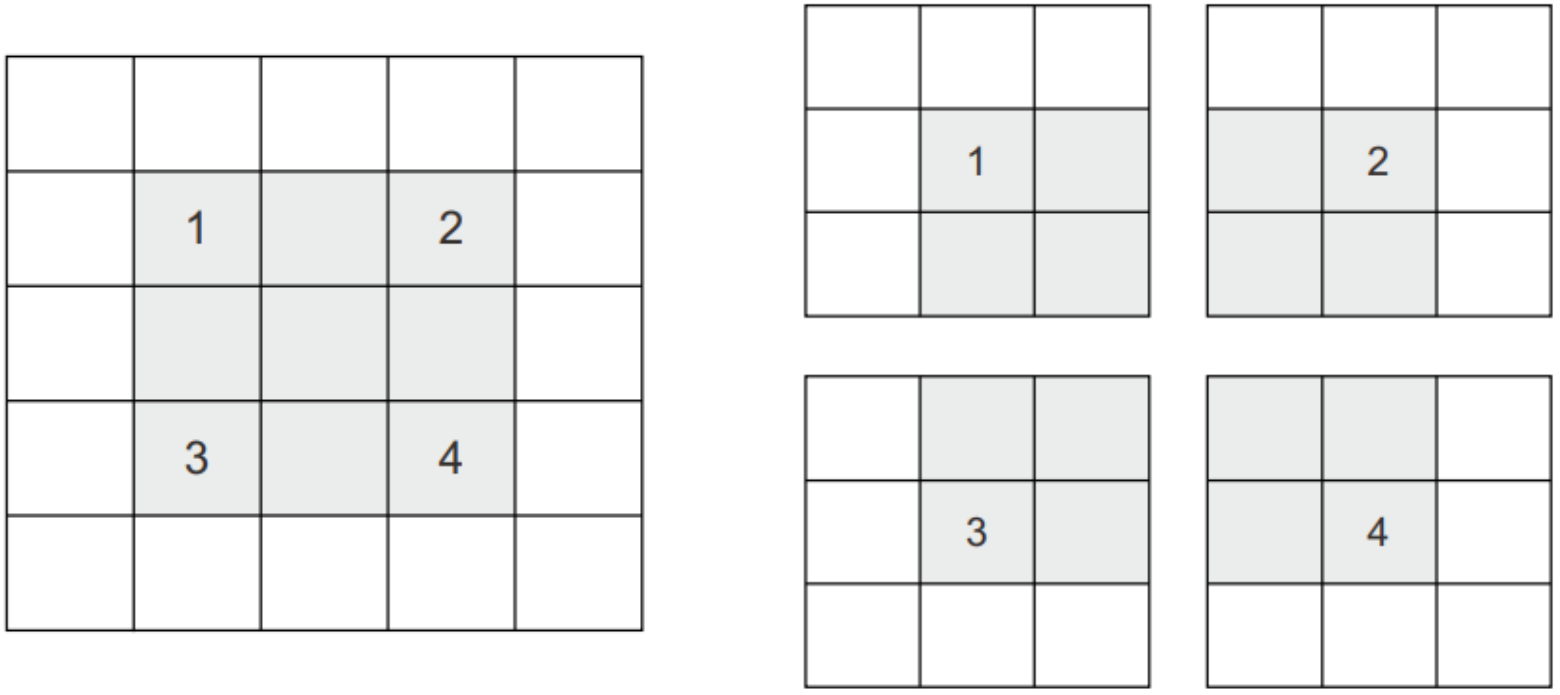


Figure 8.7 3 × 3 convolution patches with 2 × 2 strides

8.1 Introduction to convnets

▣ 8.1.2 The max-pooling operation

- In the earlier ConvNet example, the size of the feature map was reduced by half after each MaxPooling2D layer.
- For example, before the first MaxPooling2D layer, the feature map size was 26×26, but after max pooling, it became 13×13.
- Max pooling plays a role similar to strided convolution, as it forcibly downsamples the feature map.
- Max pooling extracts patches from the input feature map and outputs the maximum value for each channel.
- It is conceptually similar to convolution, but instead of applying a learned linear transformation (a convolution kernel), it uses a hardcoded maximum extraction operation.
- The main difference is that max pooling usually uses a 2×2 window and a stride of 2, which reduces the size of the feature map by half.
- In contrast, convolution typically uses a 3×3 window and a stride of 1.

8.1 Introduction to convnets

▣ 8.1.2 The max-pooling operation

- Why do we downsample the feature map in this way?
- Why not remove the max pooling layers and keep the larger feature maps instead?
- Let's test this approach and see what happens.
- The model without pooling is as follows:

Listing 8.5 An incorrectly structured convnet missing its max-pooling layers

```
inputs = keras.Input(shape=(28, 28, 1))
x = layers.Conv2D(filters=32, kernel_size=3, activation="relu")(inputs)
x = layers.Conv2D(filters=64, kernel_size=3, activation="relu")(x)
x = layers.Conv2D(filters=128, kernel_size=3, activation="relu")(x)
x = layers.Flatten()(x)
outputs = layers.Dense(10, activation="softmax")(x)
model_no_max_pool = keras.Model(inputs=inputs, outputs=outputs)
```

8.1 Introduction to convnets

8.1.2 The max-pooling operation

- Here's a summary of the model:

```
>>> model_no_max_pool.summary()
Model: "model_1"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
input_2 (InputLayer)         [(None, 28, 28, 1)]       0
_________________________________________________________________
conv2d_3 (Conv2D)            (None, 26, 26, 32)        320
_________________________________________________________________
conv2d_4 (Conv2D)            (None, 24, 24, 64)        18496
_________________________________________________________________
conv2d_5 (Conv2D)            (None, 22, 22, 128)       73856
_________________________________________________________________
flatten_1 (Flatten)          (None, 61952)             0
_________________________________________________________________
dense_1 (Dense)              (None, 10)                619530
=================================================================
Total params: 712,202
Trainable params: 712,202
Non-trainable params: 0
_________________________________________________________________
```

8.1 Introduction to convnets

▣ 8.1.2 The max-pooling operation

- What's the problem with this setup?
- There are two main issues:
  - ➢ It doesn't help the network learn the spatial hierarchy of features. The 3×3 window in the third layer only covers a 7×7 region in the original input, meaning it contains very little information about the full image. As a result, the network cannot learn higher-level patterns effectively (imagine classifying a number by only looking through a 7×7 pixel window!). The last convolutional layer's features should represent information about the entire input image.
  - ➢ The final feature map has too many elements — for example, 22×22×128 = 61,952 values, which is a lot. If this ConvNet is flattened and connected to a Dense layer with 10 units, it would create about 500,000 parameters. Such a small model would become too heavy, leading to serious overfitting.

8.1 Introduction to convnets

▣ 8.1.2 The max-pooling operation

- Max pooling is not the only way to perform downsampling.
- As we've already seen, you can also use stride in the previous convolution layers.
- Instead of taking the maximum value, you can compute the average value per channel within each input patch — this is called average pooling.
- However, max pooling usually works better than other methods.
- This is because each position in a feature map encodes whether a certain pattern or concept is present (hence the term *feature map*).
- Using the maximum value among the features tends to be more effective than using their average.
- The most relaxed form of pooling, called subsampling, first creates a downsampled feature map using convolution (without stride), and then selects the most activated feature for each small patch.
- Essentially, these methods either slide a feature-detecting window across the input (as in strided convolution) or summarize the information in patches, sometimes losing or blurring some feature details.

8.2 Training a convnet from scratch on a small dataset

▣ Training a ConvNet from Scratch on a Small Dataset

- Training an image classification model with a very small dataset is quite common.
- When you work on professional computer vision tasks, you are likely to face this situation in practice.
- A "small" sample size usually means a few thousand to tens of thousands of examples.
- As a realistic example, let's classify images of dogs and cats using a dataset of 5,000 images (2,500 dogs and 2,500 cats).
- We will use 2,000 images for training, 1,000 for validation, and 2,000 for testing.

8.2 Training a convnet from scratch on a small dataset

▣ Training a ConvNet from Scratch on a Small Dataset

- In this section, we'll look at a basic strategy to handle this problem — training a new model from scratch using a small dataset.
- With 2,000 training samples, we'll build a small ConvNet and train it without any regularization, to see its baseline performance.
- This method will reach about 70% classification accuracy.
- However, the main issue with this approach is overfitting.
- Next, we'll introduce a powerful way to reduce overfitting in computer vision — data augmentation.
- With data augmentation, the network's performance can improve to around 80–85% accuracy.

8.2 Training a convnet from scratch on a small dataset

▣ Training a ConvNet from Scratch on a Small Dataset

- Using a pretrained network to extract features (achieving about 97.5% accuracy) and then fine-tuning that pretrained network (final model reaching 98.5% accuracy).
- These three strategies
  ① training a small model from scratch,
  ② using a pretrained model for feature extraction, and
  ③ fine-tuning a pretrained model — should all be part of your toolbox when working on image classification with small datasets.

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.1 The relevance of deep learning for small-data problems

- What counts as a “sufficient number of samples” for training a model is relative.
- It depends on the size and depth of the model you’re training.
- You can’t train a complex ConvNet with only a few hundred samples.
- However, for a small and well-regularized model, a few hundred samples may be enough for a simple task.
- ConvNets are very data-efficient because they learn features that are local and translation-invariant, rather than time-dependent.
- Therefore, even with a very small image dataset, you can train a ConvNet from scratch without using domain-specific feature engineering and still achieve reasonable results.
- In this section, we will actually look at such results.

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.1 The relevance of deep learning for small-data problems

- In addition, deep learning models are highly reusable by nature.
- In other words, a model trained on a large dataset—such as an image classification or speech-to-text model—can be slightly modified and reused for a completely different task.
- Especially in computer vision, many pretrained models (usually trained on large datasets like ImageNet) are publicly available, allowing us to build powerful vision models even with very small datasets.
- This reusability is one of the greatest strengths of deep learning.

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.2 Downloading the data

- The Dogs vs. Cats dataset used here is not included in Keras.
- Before ConvNets became mainstream, this dataset was created in late 2013 as part of a Kaggle computer vision competition.
- The original dataset can be downloaded from: https://www.kaggle.com/c/dogs-vs-cats/data (You'll need a Kaggle account, but creating one is simple.)
- You can also download the dataset using the Kaggle API (see page 296, Downloading a dataset from Kaggle on Google Colab).

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.2 Downloading the data

- Downloading a Kaggle Dataset on Google Colab
  - ➢ Kaggle provides an easy-to-use API that allows you to download datasets hosted on Kaggle directly from your program.
  - ➢ For example, you can use it to download the Dogs vs. Cats dataset to your Colab notebook.
  - ➢ This API is included in the kaggle package, which is already installed in Colab.
  - ➢ You can easily download the dataset in a Colab shell by running the following command:

```
!kaggle competitions download -c dogs-vs-cats
```

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.2 Downloading the data

- Downloading a Kaggle Dataset on Google Colab
  - ➢ Only Kaggle users can use this API.
  - ➢ Before running the previous command, you need to authenticate your user account.
  - ➢ The kaggle package looks for login credentials in a JSON file located at ~/.kaggle/kaggle.json.
  - ➢ Let's create this file step by step:
    - ① First, you need to **create a Kaggle API key** and download it to your **local computer**.
      - ✓ Go to the **Kaggle website** in your web browser and log in.
      - ✓ Then go to your **Account page** and find the **API** section.
      - ✓ Click **Create New API Token**, and a file named **kaggle.json** will be generated and downloaded to your computer.

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.2 Downloading the data

- Downloading a Kaggle Dataset on Google Colab

③ Finally, create a folder at ~/.kaggle, then copy the key file into that folder (kaggle.json → ~/.kaggle/). For security, make sure **only the current user** can read this file (chmod 600).

```
!mkdir ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
```

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.2 Downloading the data

- Downloading a Kaggle Dataset on Google Colab
  - ➢ Now you can download the dataset using the following command:

```
!kaggle competitions download -c dogs-vs-cats
```

  - ➢ When downloading the dataset for the first time, you might get a ‘403 Forbidden’ error.
  - ➢ This happens because you need to agree to the competition rules before downloading the data.
  - ➢ While logged into your Kaggle account, go to: https://www.kaggle.com/c/dogs-vs-cats/rules and click the “I Understand and Accept” button.
  - ➢ You only need to agree to the rules once.

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.2 Downloading the data

- Downloading a Kaggle Dataset on Google Colab
  - ➢ The training data comes as a compressed file named dogs-vs-cats.zip.
  - ➢ After unzipping it, another file called train.zip will be created — unzip that as well.
  - ➢ When unzipping, you can add the -qq option to suppress output messages.

```
!unzip -qq train.zip
```

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.2 Downloading the data

- Downloading a Kaggle Dataset on Google Colab
  - ➢ The images in the dataset are color JPEG files with medium resolution.
  - ➢ You can see a few sample images in Figure 8-8.

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.2 Downloading the data



Figure 8.8 Samples from the Dogs vs. Cats dataset. Sizes weren't modified: the samples come in different sizes, colors, backgrounds, etc.

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.2 Downloading the data

- As expected, in the 2013 Kaggle Dogs vs. Cats competition, the winners used ConvNets.
- The top-performing models achieved around 95% accuracy.
- In the next section, we'll train a model using less than 10% of the data used by those participants and still reach a very similar accuracy.

8.2 Training a convnet from scratch on a small dataset

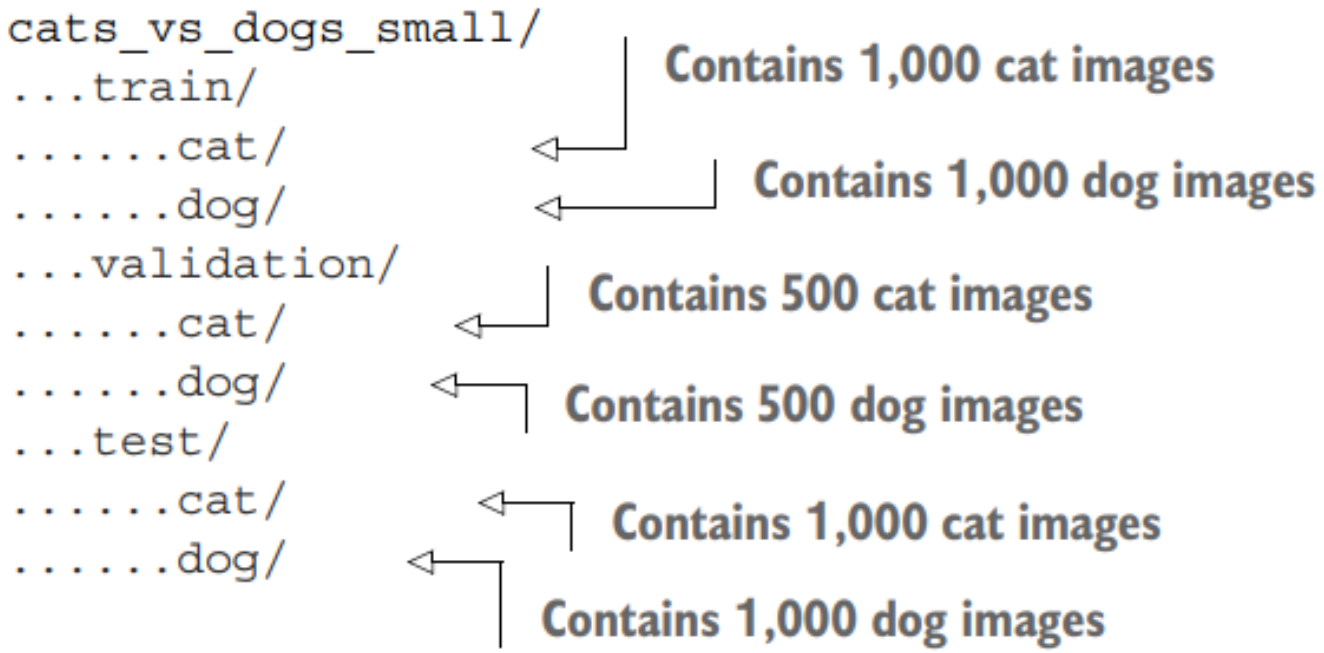
▣ 8.2.2 Downloading the data

- This dataset contains 25,000 images of dogs and cats (12,500 per class) and is 543 MB in compressed form.
- After downloading and unzipping, we'll create a new dataset with three subsets:
  - A training set with 1,000 samples per class
  - A validation set with 500 samples per class
  - A test set with 1,000 samples per class
- As you gain more experience in this field, you'll often encounter image datasets with only a few thousand samples, not millions.
- Having more data usually makes the problem easier, but for learning purposes, working with smaller datasets is better.

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.2 Downloading the data

- The subsampled dataset we will work with will have the following directory structure:



8.2 Training a convnet from scratch on a small dataset

▣ 8.2.2 Downloading the data

- Let's make it happen in a couple calls to shutil

Listing 8.6 Copying images to training, validation, and test directories

```
import os, shutil, pathlib

original_dir = pathlib.Path("train")
new_base_dir = pathlib.Path("cats_vs_dogs_small")

def make_subset(subset_name, start_index, end_index):
    for category in ("cat", "dog"):
        dir = new_base_dir / subset_name / category
        os.makedirs(dir)
        fnames = [f"{category}.{i}.jpg"
                  for i in range(start_index, end_index)]
        for fname in fnames:
            shutil.copyfile(src=original_dir / fname,
                            dst=dir / fname)

make_subset("train", start_index=0, end_index=1000)
make_subset("validation", start_index=1000, end_index=1500)
make_subset("test", start_index=1500, end_index=2500)
```

Path to the directory where the original dataset was uncompressed

Directory where we will store our smaller dataset

Create the training subset with the first 1,000 images of each category.

Create the validation subset with the next 500 images of each category.

Create the test subset with the next 1,000 images of each category.

Utility function to copy cat (and dog) images from index start_index to index end_index to the subdirectory new_base_dir/{subset_name}/cat (and /dog). The "subset_name" will be either "train", "validation", or "test".

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.3 Building the model

- We'll reuse the same general model structure as in the first example.
- It's a ConvNet made by stacking Conv2D (with ReLU activation) and MaxPooling2D layers alternately.
- Since the images are larger and the problem is more complex, we'll make the model a bit bigger.
- We'll add one more pair of Conv2D and MaxPooling2D layers.
- This helps keep the feature map size smaller before the Flatten layer, so the model size doesn't grow too large.
- Starting from an input of 180×180, the model reduces it to a 7×7 feature map before flattening.
- The depth of the feature map gradually increases (from 32 up to 256), while its spatial size decreases (from 180×180 to 7×7).
- This is a typical pattern seen in almost all ConvNets.

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.3 Building the model

- Since this is a binary classification problem, the model ends with a single unit (a Dense layer with size 1) using a sigmoid activation function.
- This unit encodes the probability that the sample the model sees belongs to one of the two classes.
- One final small detail: the model begins with a Rescaling layer.
- This layer scales the image input (originally in the range [0, 255]) to the range [0, 1].

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.3 Building the model

Listing 8.7 Instantiating a small convnet for dogs vs. cats classification

```
from tensorflow import keras
from tensorflow.keras import layers
inputs = keras.Input(shape=(180, 180, 3))
x = layers.Rescaling(1./255)(inputs)
x = layers.Conv2D(filters=32, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=64, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=128, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=256, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=256, kernel_size=3, activation="relu")(x)
x = layers.Flatten()(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs=inputs, outputs=outputs)
```

The model expects RGB images of size 180 × 180.

Rescale inputs to the [0, 1] range by dividing them by 255.

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.3 Building the model

```
>>> model.summary()
Model: "model_2"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
input_3 (InputLayer)         [(None, 180, 180, 3)]     0
_________________________________________________________________
rescaling (Rescaling)        (None, 180, 180, 3)       0
_________________________________________________________________
conv2d_6 (Conv2D)            (None, 178, 178, 32)      896
_________________________________________________________________
max_pooling2d_2 (MaxPooling2 (None, 89, 89, 32)        0
_________________________________________________________________
conv2d_7 (Conv2D)            (None, 87, 87, 64)        18496
_________________________________________________________________
max_pooling2d_3 (MaxPooling2 (None, 43, 43, 64)        0
_________________________________________________________________
conv2d_8 (Conv2D)            (None, 41, 41, 128)       73856
_________________________________________________________________
max_pooling2d_4 (MaxPooling2 (None, 20, 20, 128)       0
_________________________________________________________________
conv2d_9 (Conv2D)            (None, 18, 18, 256)       295168
_________________________________________________________________
max_pooling2d_5 (MaxPooling2 (None, 9, 9, 256)         0
_________________________________________________________________
conv2d_10 (Conv2D)           (None, 7, 7, 256)         590080
_________________________________________________________________
flatten_2 (Flatten)          (None, 12544)             0
_________________________________________________________________
dense_2 (Dense)              (None, 1)                 12545
=================================================================
Total params: 991,041
Trainable params: 991,041
Non-trainable params: 0
_________________________________________________________________
```

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.3 Building the model

- For the compilation step, we'll use the RMSprop optimizer, just like before.
- Since the model ends with a single sigmoid unit, we'll use binary cross-entropy as the loss function.

Listing 8.8 Configuring the model for training

```
model.compile(loss="binary_crossentropy",
              optimizer="rmsprop",
              metrics=["accuracy"])
```

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.4 Data preprocessing

- Before being fed into the network, the data must be properly preprocessed into floating-point tensors.
- Since the data is currently in JPEG format, the general steps for preparing it for the network are as follows:
  ① Read the image files.
  ② Decode the JPEG content into RGB pixel values.
  ③ Convert the data into floating-point tensors.
  ④ Resize all images to the same size (here, we'll use 180×180).
  ⑤ Group them into batches (each batch containing 32 images).

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.4 Data preprocessing

- Although this process may seem a bit complicated, Keras provides a utility that handles these steps automatically.
- Specifically, Keras offers the image_dataset_from_directory() function.
- This function automatically reads image files from disk, converts them into preprocessed tensors, and organizes them into batches ready for training.
- We'll use this function here.
- When you call image_dataset_from_directory(directory), it first looks for subdirectories within the given directory.
- Each subdirectory is assumed to contain images for one class.
- Then, the function indexes all image files within those subdirectories.
- Finally, it reads the files, shuffles their order, decodes them into tensors, resizes them to a uniform size, and returns them as a tf.data.Dataset object grouped into batches.

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.4 Data preprocessing

Listing 8.9 Using `image_dataset_from_directory` to read images

```
from tensorflow.keras.utils import image_dataset_from_directory

train_dataset = image_dataset_from_directory(
    new_base_dir / "train",
    image_size=(180, 180),
    batch_size=32)
validation_dataset = image_dataset_from_directory(
    new_base_dir / "validation",
    image_size=(180, 180),
    batch_size=32)
test_dataset = image_dataset_from_directory(
    new_base_dir / "test",
    image_size=(180, 180),
    batch_size=32)
```

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.4 Data preprocessing

- **Understanding the tf.data.Dataset Object**
  - TensorFlow provides the tf.data API, which allows you to build efficient input pipelines for training models.
  - The core class is tf.data.Dataset.
  - A Dataset object is an iterator, meaning it can be used in a for loop and typically returns batches of input data and labels.
  - You can pass a Dataset object directly to the Keras model's fit() method.
  - The Dataset class handles many essential functions that are difficult to implement manually.
  - In particular, the data prefetching feature allows the next batch to be prepared while the previous one is being processed, enabling the model to train without pauses.

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.4 Data preprocessing

- **Understanding the tf.data.Dataset Object**
  - The Dataset class also provides a functional-style API for creating and manipulating datasets.
  - Here's a simple example:
    - Let's create a Dataset object using random NumPy arrays.
    - We'll generate 1,000 samples,
    - Each sample will be a vector of size 16.

```
import numpy as np
import tensorflow as tf
random_numbers = np.random.normal(size=(1000, 16))
dataset = tf.data.Dataset.from_tensor_slices(random_numbers)
```

The from_tensor_slices() class method can be used to create a Dataset from a NumPy array, or a tuple or dict of NumPy arrays.

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.4 Data preprocessing

- **Understanding the tf.data.Dataset Object**
  - ➢ At first, our dataset just yields single samples:

```
>>> for i, element in enumerate(dataset):
>>>     print(element.shape)
>>>     if i >= 2:
>>>         break
(16,)
(16,)
(16,)
```

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.4 Data preprocessing

- **Understanding the tf.data.Dataset Object**
  - ➢ We can use the .batch() method to batch the data:

```
>>> batched_dataset = dataset.batch(32)
>>> for i, element in enumerate(batched_dataset):
>>>     print(element.shape)
>>>     if i >= 2:
>>>         break
(32, 16)
(32, 16)
(32, 16)
```

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.4 Data preprocessing

- **Understanding the tf.data.Dataset Object**
  - ➢ The .map() method is especially useful and frequently used.
  - ➢ For example, let's transform the element size in our dataset from (16,) to (4,).

```
>>> reshaped_dataset = dataset.map(lambda x: tf.reshape(x, (4, 4)))
>>> for i, element in enumerate(reshaped_dataset):
>>>     print(element.shape)
>>>     if i >= 2:
>>>         break
(4, 4)
(4, 4)
(4, 4)
```

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.4 Data preprocessing

- Let's now train a model on this dataset.
- We'll monitor validation metrics by passing a separate Dataset object through the validation_data argument in the fit() method.
- We'll also use the ModelCheckpoint callback to save the model at the end of each epoch.
- In the callback, specify the file path and set the parameters save_best_only=True and monitor='val_loss'.
- During training, the callback will save a new file only when the val_loss value improves (i.e., becomes lower than before).
- This way, the saved file will always contain the best-performing model on the validation data.
- As a result, once overfitting starts, you won't need to retrain the model for many epochs again.
- You can simply load the model directly from the saved file.

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.4 Data preprocessing

Listing 8.10 Displaying the shapes of the data and labels yielded by the `Dataset`

```
>>> for data_batch, labels_batch in train_dataset:
>>>     print("data batch shape:", data_batch.shape)
>>>     print("labels batch shape:", labels_batch.shape)
>>>     break
data batch shape: (32, 180, 180, 3)
labels batch shape: (32,)
```

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.4 Data preprocessing

Listing 8.11 Fitting the model using a `Dataset`

```
callbacks = [
    keras.callbacks.ModelCheckpoint(
        filepath="convnet_from_scratch.keras",
        save_best_only=True,
        monitor="val_loss")
]

history = model.fit(
    train_dataset,
    epochs=30,
    validation_data=validation_dataset,
    callbacks=callbacks)
```

8.2.4 Data preprocessing

- Let's plot the loss and accuracy of the model over the training and validation data during training.

Listing 8.12 Displaying curves of loss and accuracy during training

```
import matplotlib.pyplot as plt
accuracy = history.history["accuracy"]
val_accuracy = history.history["val_accuracy"]
loss = history.history["loss"]
val_loss = history.history["val_loss"]
epochs = range(1, len(accuracy) + 1)
plt.plot(epochs, accuracy, "bo", label="Training accuracy")
plt.plot(epochs, val_accuracy, "b", label="Validation accuracy")
plt.title("Training and validation accuracy")
plt.legend()
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.legend()
plt.show()
```

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.4 Data preprocessing

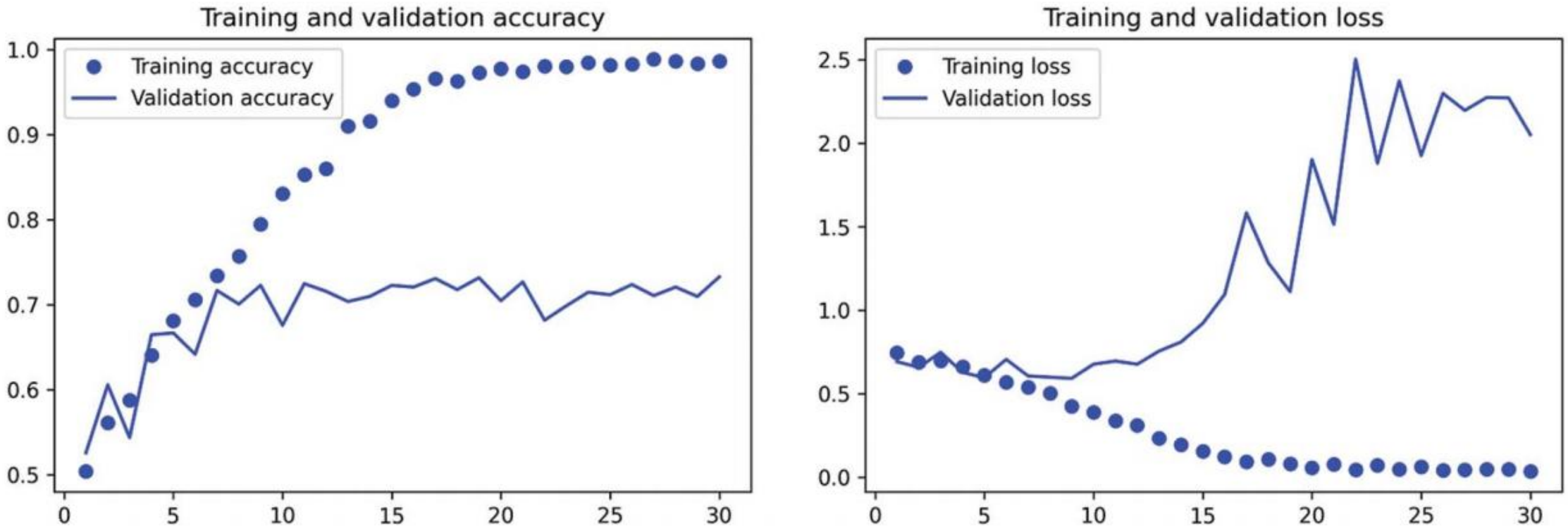


Figure 8.9 Training and validation metrics for a simple convnet

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.4 Data preprocessing

- This graph shows the characteristics of overfitting.
- The training accuracy increases almost linearly over time, reaching nearly 100%.
- In contrast, the validation accuracy peaks at around 75%.
- The validation loss reaches its minimum after just a few epochs and then stops improving.
- Meanwhile, the training loss continues to decrease linearly as training progresses.

8.2 Training a convnet from scratch on a small dataset

8.2.4 Data preprocessing

- Let's check the test accuracy.
- We'll reload the model from its saved file to evaluate it as it was before it started overfitting.

Listing 8.13 Evaluating the model on the test set

```
test_model = keras.models.load_model("convnet_from_scratch.keras")
test_loss, test_acc = test_model.evaluate(test_dataset)
print(f"Test accuracy: {test_acc:.3f}")
```

- We get a test accuracy of 69.5%.

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.4 Data preprocessing

- Because the number of training samples (2,000) is relatively small, overfitting becomes the most important issue.
- We've already learned several basic techniques to reduce overfitting, such as dropout and weight decay (L2 regularization).
- Here, since this task involves computer vision, we'll try a new and very common method used when working with images — data augmentation.

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.5 Using data augmentation

- Overfitting occurs when the number of training samples is too small to train a model that generalizes well to new data.
- If we had an unlimited amount of data, the model could learn all possible aspects of the data distribution.
- Data augmentation is a technique that creates more training data from existing samples.
- It works by generating modified versions of images through various random transformations.
- The goal is to ensure the model never sees the exact same image twice during training.
- By learning from many different variations of the data, the model can generalize better.

8.2.5 Using data augmentation

- In Keras, you can add several data augmentation layers at the beginning of your model.
- For example,
  - The following Sequential model performs several random image transformations.
  - We'll place this model just before the Rescaling layer of the ConvNet.

Listing 8.14 Define a data augmentation stage to add to an image model

```
data_augmentation = keras.Sequential(
    [
        layers.RandomFlip("horizontal"),
        layers.RandomRotation(0.1),
        layers.RandomZoom(0.2),
    ]
)
```

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.5 Using data augmentation

- There are many more augmentation layers available (for details, see the Keras documentation).
- Here's a quick look at the code example:
  - RandomFlip("horizontal"): randomly flips 50% of the images horizontally.
  - RandomRotation(0.1): rotates the input image by a random value within the range [-10%, +10%] of a full circle (approximately -36° to +36°).
  - RandomZoom(0.2): randomly zooms the image in or out by up to ±20%.

8.2 Training a convnet from scratch on a small dataset

8.2.5 Using data augmentation

- Let's look at the augmented images.

Listing 8.15 Displaying some randomly augmented training images

```
plt.figure(figsize=(10, 10))
for images, _ in train_dataset.take(1):
    for i in range(9):
        augmented_images = data_augmentation(images)
        ax = plt.subplot(3, 3, i + 1)
        plt.imshow(augmented_images[0].numpy().astype("uint8"))
        plt.axis("off")
```

We can use take(N) to only sample *N* batches from the dataset. This is equivalent to inserting a break in the loop after the *N*th batch.

Apply the augmentation stage to the batch of images.

Display the first image in the output batch. For each of the nine iterations, this is a different augmentation of the same image.

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.5 Using data augmentation



Figure 8.10 Generating variations of a very good boy via random data augmentation

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.5 Using data augmentation

- When training a new model with data augmentation, the same input data is never fed twice to the model.
- However, since the augmented images are generated from a limited number of original images, there is still a high correlation between the samples.
- In other words, augmentation does not create entirely new information — it just rearranges existing data.
- Therefore, data augmentation alone may not be enough to completely eliminate overfitting.
- To further reduce overfitting, we'll add a Dropout layer before the densely connected classifier.

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.5 Using data augmentation

- One last thing to know about random image augmentation layers: like Dropout layers, they are active only during training — they do not operate when calling predict() or evaluate().
- In other words, when evaluating the model, it behaves as if there were no data augmentation or dropout applied.

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.5 Using data augmentation

Listing 8.16 Defining a new convnet that includes image augmentation and dropout

```
inputs = keras.Input(shape=(180, 180, 3))
x = data_augmentation(inputs)
x = layers.Rescaling(1./255)(x)
x = layers.Conv2D(filters=32, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=64, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=128, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=256, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=256, kernel_size=3, activation="relu")(x)
x = layers.Flatten()(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs=inputs, outputs=outputs)

model.compile(loss="binary_crossentropy",
              optimizer="rmsprop",
              metrics=["accuracy"])
```

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.5 Using data augmentation

- Let's train the model using data augmentation and dropout.
- Because we expect overfitting to occur much later during training, we will train for three times as many epochs—one hundred.

Listing 8.17 Training the regularized convnet

```
callbacks = [
    keras.callbacks.ModelCheckpoint(
        filepath="convnet_from_scratch_with_augmentation.keras",
        save_best_only=True,
        monitor="val_loss")
]
history = model.fit(
    train_dataset,
    epochs=100,
    validation_data=validation_dataset,
    callbacks=callbacks)
```

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.5 Using data augmentation

- Let's plot the results again: see figure 8.11.

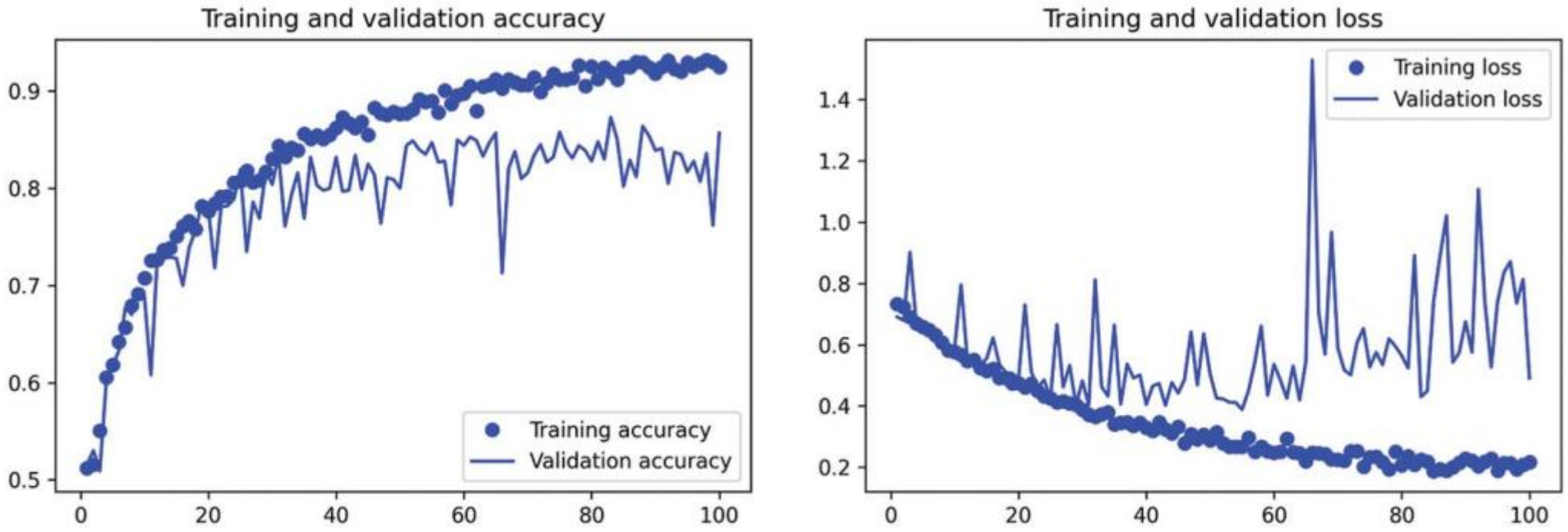


Figure 8.11 Training and validation metrics with data augmentation

8.2 Training a convnet from scratch on a small dataset

▣ 8.2.5 Using data augmentation

- Thanks to data augmentation and dropout, we start overfitting much later, around epochs 60–70 (compared to epoch 10 for the original model).
- The validation accuracy ends up consistently in the 80–85% range — a big improvement over our first try.

8.2 Training a convnet from scratch on a small dataset

8.2.5 Using data augmentation

- Let's check the test accuracy.

Listing 8.18 Evaluating the model on the test set

```
test_model = keras.models.load_model(
    "convnet_from_scratch_with_augmentation.keras")
test_loss, test_acc = test_model.evaluate(test_dataset)
print(f"Test accuracy: {test_acc:.3f}")
```

- We get a test accuracy of 83.5%.



Q & A

Prof. Chang-Hee Han